Pipelines, Speculation, and Address Translation

Microarchitecture Experiments on Two Intel Generations

Varunaditya Venkatesan
Unity ID: vvenka25
ECE 592 — Homework 3, Track B
North Carolina State University, Fall 2026
November 7, 2026

GitHub source and evidence:
https://github.com/vvenka25/ECE-592-Project-3
Overleaf project:
https://www.overleaf.com/project/6aeeb80e5d4ccedd231a327b

Selected exercises: Q1, Q2, Q3, Q4, Q5, Q8 — 105 points.

Question	Experiment	Points	Question	Experiment	Points
Q1	Pipelining	10	Q4	ROB capacity	30
Q2	Out-of-order execution	10	Q5	Integer physical registers	35
Q3	Branch prediction	10	Q8	TLB-miss cost	10

Abstract. We examine six microarchitectural properties on Charnwood's Core i7-6700 (Skylake) and Artemisia's Xeon Gold 5420+ (Sapphire Rapids), with paired hardware-debug SGX measurements on Artemisia. Fixed-work integer multiplication shows approximately three active core cycles of dependency latency and a one-cycle initiation interval on both CPUs. A blocked-older/ready-younger arithmetic experiment, with dependency and fence controls, supports out-of-order execution. Balanced shuffled branch outcomes cost substantially more than regular outcomes, while a branchless control remains nearly unchanged. Repeated ROB-spacing sweeps place effective overlap loss at N=220–224 on Charnwood and N=492–500 on Artemisia. Scaled-index LEA writes expose earlier integer-GPR destination-pressure regions, approximately N=128–144 and N=200–216; the evidence does not establish total physical register-file size. For native L1 data-TLB misses served by the STLB, cache/event-controlled baseline subtraction gives about nine and seven additional active core cycles, respectively. The enclave retains the principal dependency, branch and capacity trends, but its protected TLB-event coverage is unqualified. All results use retained raw measurements; process repetitions, timing units, controls and attribution limits remain explicit.

---

1 Experimental setup and common measurements

1.1 Machines and placement

Campaign inventories identify the two CPUs and collection-time configuration (Table 1). Every selected question has native measurements on both course machines, as required [1].

	Charnwood	Artemisia
CPU	Intel Core i7-6700	Intel Xeon Gold 5420+
Microarchitecture	Skylake	Sapphire Rapids
Topology	1 socket, 4 cores, 8 logical CPUs	2 sockets, 56 cores, 112 logical CPUs; CPU 68 offline
NUMA domains	1	2
New-suite placement	CPU 1, node 0; sibling 5 online	CPU 12, node 0; sibling 68 offline
SMT control	Private inherited core-scheduling group	No online sibling; before/after activity gate
Compiler	GCC 13.3.0, `-O2`	GCC 11.5.0, `-O2`
SGX access	No current/legacy SGX device path	Hardware-debug SGX with system PSW runtime

Table 1: Recorded configuration. Charnwood's controller is on CPU 0; new-suite controllers use another physical core. These are collection-time settings, so reproduction must recheck topology and affinity.

Affinity precedes allocation. Native memory experiments request node 0 and 4 KiB backing. The final protocol requires full RSS, no transparent huge pages and read-only per-page NUMA queries before/after timing; it requests no page migration [9]. Earlier campaigns retain their recorded checks. These host audits do not verify physical EPC placement. Charnwood's core group excludes unrelated userspace co-scheduling, with kernel/interrupt contexts still possible [8].

Preserved Q4 is a distinct configuration. Artemisia's completed Q4 runs used CPU 1/node 0, with sibling 57 activity recorded as uncontrolled. Charnwood Q4 also used CPU 1/node 0; an external private core group was applied, although the legacy manifest says uncontrolled. Its external-control records identify the actual group; the legacy helper mask also permits CPU 1. Neither history is rewritten as equivalent to the newer CPU-12 protocol.

1.2 Samples, work and statistics

Each treatment keeps 15 timed samples and two fresh-process repetitions; job and parameter orders are shuffled with recorded seeds. The randomized pointer cycles cover the full requested footprint and continue between samples. Full-ring or full-pattern warm-up and a short kernel warm-up precede timing. All primary observations remain, including recorded ordinary faults and context switches. Solid circles denote process 1, dashed squares process 2 unless a caption specifies another control style. Shading or error bars show observed inclusive p10–p90, not confidence intervals. Tables show separate process medians as **R1** / **R2**; no host, environment or repetition is pooled.

A sample records host TSC ticks. Q1 divides by $32I$ IMULs; Q2, Q3 and Q8 divide by I blocks, decisions and loads; Q4 and Q5 divide by $2I$ loads. TSC ticks are not active core cycles. We neither convert ticks using nominal MHz nor assume equal tick duration across machines. Native PMU core cycles are labeled separately. Counter brackets include slightly more host-user work than the TSC interval and require at least 98% scheduled coverage for qualification.

1.3 SGX background and same-machine comparison

Intel SGX provides enclaves: application regions whose code and data receive hardware-enforced isolation from ordinary host software. SGX applications separate trusted enclave code from an untrusted host, using explicit enclave calls to cross that boundary [2]. Here the enclave allocates and initializes its private arena and executes the assembly kernel; the host sets affinity, creates the enclave, invokes initialization/warm-up, reads the timer and counters, and emits raw data. Artemisia used hardware-debug enclaves (`SGX_DEBUG_FLAG`, `DisableDebug=0`), not simulation. Debug mode permits inspection and is a development configuration [2]. The user previously confirmed Intel SampleEnclave with `SGX_MODE=HW SGX_DEBUG=1`; that console output was not saved. Retained custom-enclave semantic and timing checks establish the benchmark's own hardware execution separately.

Comparing native and SGX on the *same* Artemisia CPU asks whether the same instruction stream exposes the same behavior, and whether its timing or transition changes. Within each paired campaign, the kernel bytes, input pattern or ring geometry, footprint, seed, filler/chain choices, iteration count, sample count and CPU/node request match. Both environments use host LFENCE/RDTSCP/LFENCE boundaries around one invocation. The SGX timed interval includes ECALL entry/exit and marshalling; enclave creation, allocation and warm-up are outside. Native arenas use anonymous mmap; protected arenas use enclave allocation, with Q8 explicitly page-aligned in both. Differences in runtime transitions, protected memory placement, cache/translation state, frequency or interruptions are possible explanations, not separately measured causes of a timing difference.

Enclave setting	Q1/Q2/Q3/Q5/Q8 suite	Preserved Q4
`HeapMaxSize`	`0x18000000` (384 MiB)	`0x30000000` (768 MiB)
`StackMaxSize`	`0x40000` (256 KiB)	`0x40000` (256 KiB)
`TCSNum / TCSPolicy`	1 / 1	1 / 1
`ProdID / ISVSVN / DisableDebug`	0 / 0 / 0	0 / 0 / 0
`MiscSelect / MiscMask`	0 / `0xFFFFFFFF`	0 / `0xFFFFFFFF`

Table 2: Configuration taken from the benchmark XML and campaign source snapshots. SDK path: `/opt/intel/sgxsdk`; retained runtime/PSW packages report version 2.23.100.2.

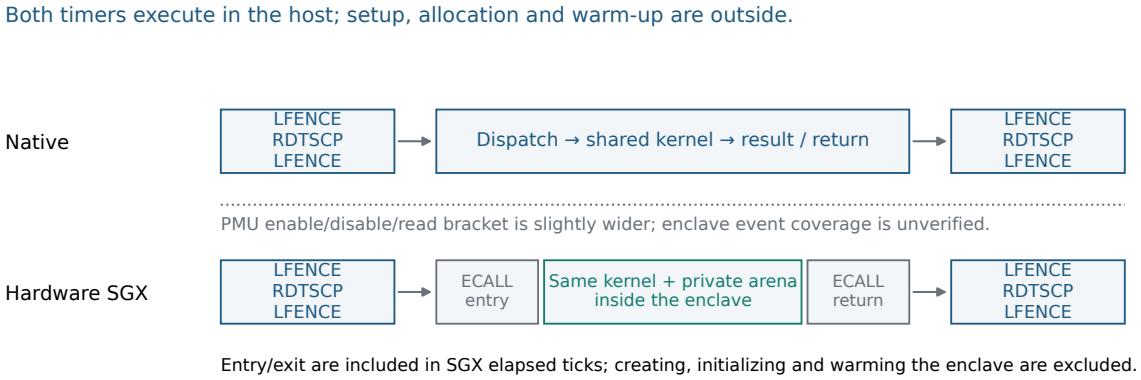


Figure 1: The common host timer encloses the native invocation or the ECALL. Protected allocation and physical EPC placement differ from native mmap; equal kernel bytes do not make those memory environments identical.

```
unsigned a,b;_mm_lfence();uint64_t start=__rdtscp(&a);_mm_lfence();
uint64_t value=invoke(mode,p,iters);
_mm_lfence();uint64_t end=__rdtscp(&b);_mm_lfence();
```

2 Q1: confirm pipelining (10 points)

2.1 Method and observable signal

A pipelined execution resource can start a new operation before the previous operation produces its dependent result. We therefore measure dependency latency and initiation interval with fixed instruction work. Each loop executes 32 explicit `imul $3,reg,reg` instructions, distributed over one, two or four independent 64-bit GPR chains. Odd starting values and modulo-2^{64} arithmetic give checked, observable results; the compiler cannot replace the assembly IMULs with LEAs. With one chain, every multiply waits for its own prior result. Additional chains make ready multiplies available without changing the total multiply count.

Each sample contains 100,000 loops (3.2 million IMULs). Register initialization, checksum generation and invocation are included in the call, so zero-work controls measure their fixed contribution. We expect per-IMUL time to fall until execution throughput limits further improvement. That expectation alone does not distinguish pipelining from replicated execution units.

```
.Lloop_tb_q1_4:
imul $3,%r8,%r8
imul $3,%r9,%r9
imul $3,%r10,%r10
imul $3,%r11,%r11
# The four-instruction group repeats eight times (32 IMULs).
dec %rsi
jnz .Lloop_tb_q1_4
```

Listing 1: Actual four-chain loop excerpt from `tb_q1_4`. The shown group occurs eight times per loop; the comment marks omitted repetition.

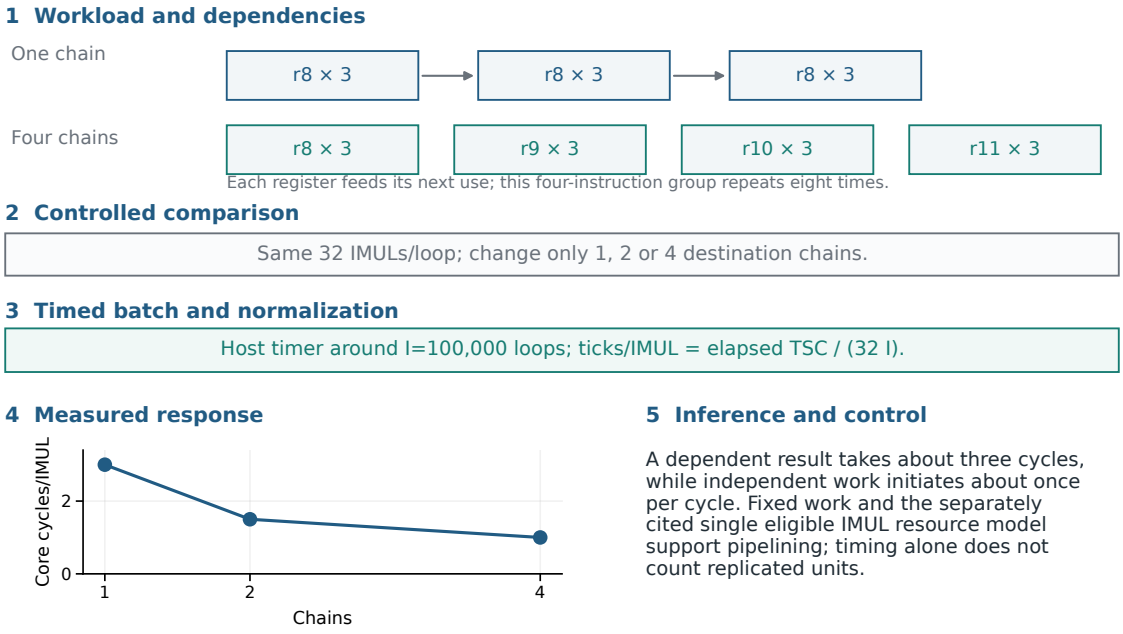


Figure 2: Q1 procedure. The small measured plot uses Artemisia native process 1. The arrows denote dependencies, not an instruction-by-instruction hardware trace.

2.2 Native results, paired enclave result and inference

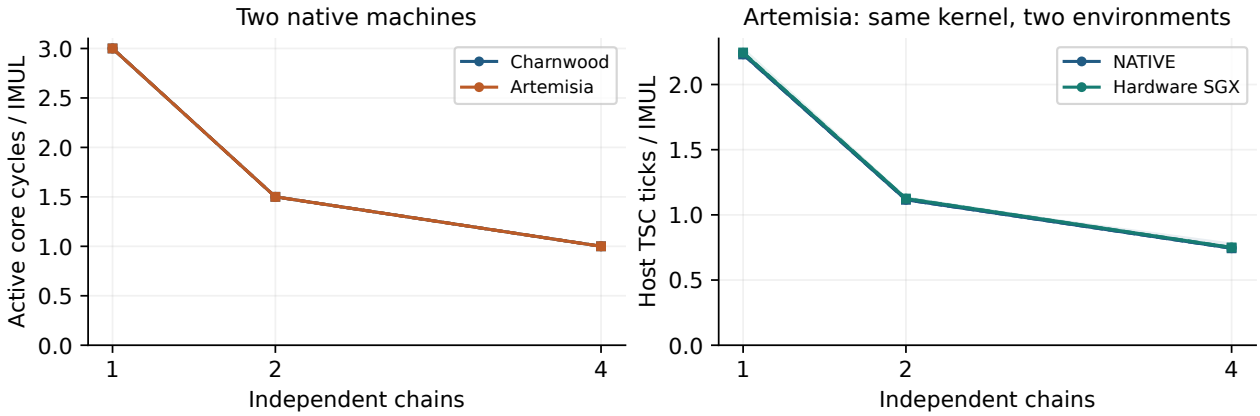


Figure 3: Q1 fixed-work results. Left: native active core cycles per IMUL on both machines. Right: Artemisia host TSC ticks per IMUL in native and hardware SGX. Process repetitions and observed p10–p90 remain separate.

Chains	Charnwood cycles/IMUL	Artemisia cycles/IMUL
1	3.0006 / 3.0006	3.0005 / 3.0005
2	1.5003 / 1.5003	1.5003 / 1.5003
4	1.0003 / 1.0003	1.0002 / 1.0002

Table 3: Native Q1 process medians in active core cycles/IMUL (R1 / R2).

Chains	Artemisia native ticks/IMUL	Artemisia SGX ticks/IMUL
1	2.2314 / 2.2308	2.2435 / 2.2456
2	1.1156 / 1.1155	1.1249 / 1.1251
4	0.7440 / 0.7435	0.7500 / 0.7502

Table 4: Paired Artemisia Q1 process medians in TSC ticks/IMUL (R1 / R2).

Figure 3 and Table 3 show approximately 3.00, 1.50 and 1.00 active core cycles/IMUL for one, two and four chains, respectively. The one-chain latency-dominated cost is about three times the sufficiently independent initiation interval. Zero-work medians are 76 ticks/invocation on Charnwood, 64 natively on Artemisia and 11,046 in its enclave (process 1); no overhead subtraction is applied. That fixed cost is small per operation in a 3.2-million-IMUL batch.

Alternative explanation. Multiple multiplier units could also speed independent work. As a separately labeled reference model, LLVM assigns the selected 64-bit immediate/register IMUL to a single eligible port-1 resource on Skylake and Sapphire Rapids [3, 4]. Combined with that resource bound, the measured latency/throughput gap supports pipelined multiplication. We have not experimentally counted the units or measured whole-processor pipeline depth; the model is supporting evidence rather than a measured specification.

Artemisia SGX preserves the chain-count trend. Its higher ticks coexist with larger included invocation costs; neither protected arithmetic latency nor a change in multiplier design is isolated, since enclave PMU coverage is unqualified.

Evidence and reconstruction. `results/q1-charnwood-private/` and `results/q1-artemisia/`: 120 and 240 raw observations, including zero-work controls. Source: `trackb/gen_kernels.py`; reconstruction: `report/make_report_artifacts.py`.

3 Q2: confirm out-of-order execution (10 points)

3.1 Method and observable signal

We place an *older unready arithmetic instruction* before younger ready work. A pointer load feeds an older IMUL; a later chain of K IMULs uses R9 and a ready XOR source. Program-order issue would have to wait at the older unready multiply. Out-of-order issue can execute the younger independent chain while the load and older multiply are pending.

The dependency control changes the XOR source from ready R11 to the older result in RAX. A third variant inserts LFENCE before the younger work. *Every* variant also has a common LFENCE at the block end, so younger arithmetic cannot hide behind the next iteration's load. This execution boundary uses Intel's LFENCE ordering semantics [5]. The same random pointer traversal runs with 16 KiB hot and 256 MiB cold footprints. K=0 exposes baseline/fence cost; K=64 supplies substantial younger work. Each sample measures 100,000 blocks.

```
.Lloop_tb_q2_overlap_64:
mov (%r8),%r8
imul $3,%r8,%rax
xor %r11,%r9
imul $3,%r9,%r9
# 63 further younger IMULs are omitted here.
xor %rax,%r10
lfence
dec %rsi
jnz .Lloop_tb_q2_overlap_64
```

Listing 2: Actual overlap kernel `tb_q2_overlap_64`. The dependent variant reads RAX in the XOR; the fenced variant inserts LFENCE after the older IMUL.

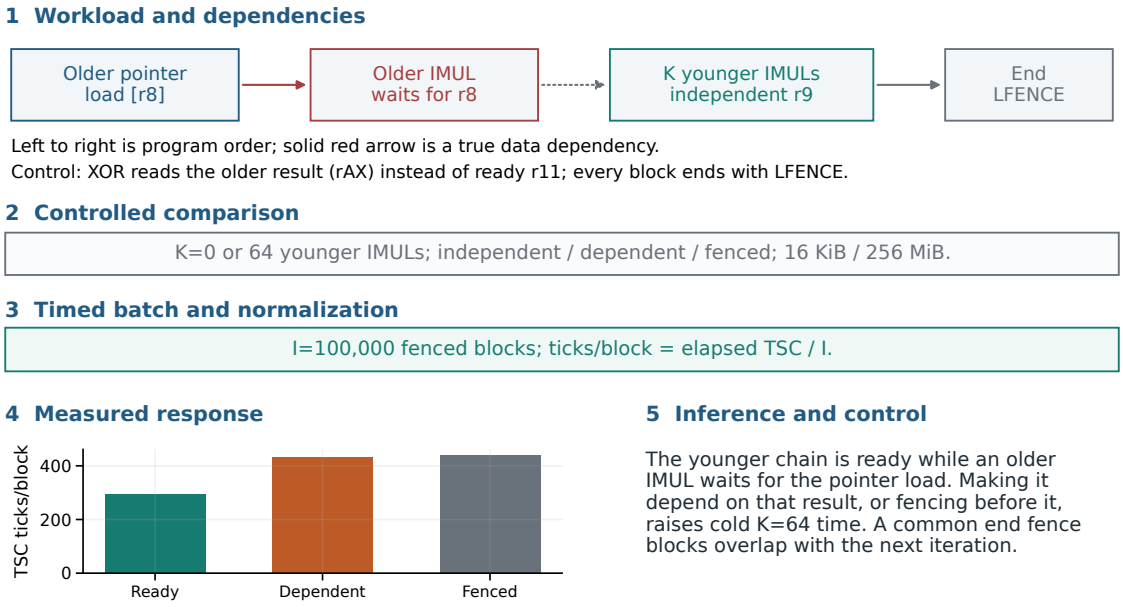


Figure 4: Q2 tests bypass of an older blocked operation, beyond independent-stream throughput. The measured response is Artemisia native process 1 at cold K=64.

3.2 Native results, paired enclave result and inference

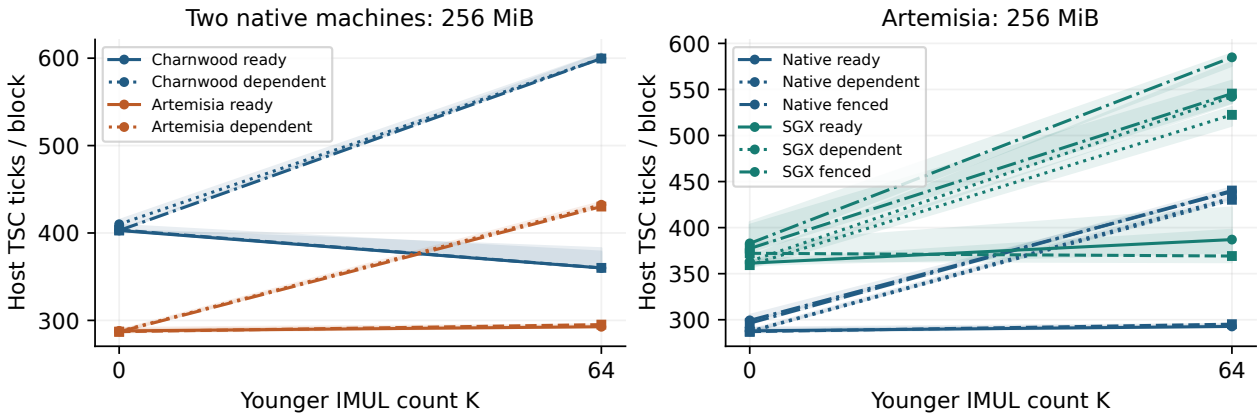


Figure 5: Q2 cold-data comparisons. Dependence/fence variants are named in the legend; control styles differ from the general process-style convention. Only measured K=0 and K=64 conditions are joined; no intermediate measurements are implied.

Condition	Charnwood native	Artemisia native	Artemisia SGX
Cold, independent, K=0	402.88 / 403.50	287.97 / 287.11	361.41 / 372.15
Cold, independent, K=64	359.96 / 360.08	292.72 / 295.18	387.02 / 369.17
Cold, dependent, K=64	599.72 / 599.63	432.31 / 430.24	542.05 / 522.43
Cold, fenced, K=64	598.99 / 602.43	439.04 / 440.26	584.86 / 545.50
Hot, independent, K=64	184.76 / 184.82	153.91 / 153.84	154.93 / 154.87
Hot, dependent, K=64	189.74 / 189.75	158.44 / 158.58	159.52 / 159.50

Table 5: Q2 medians in TSC ticks/block, R1 / R2. Hot is 16 KiB; cold is 256 MiB.

In Figure 5 and Table 5, cold K=64 independent younger work takes about 360 ticks/block on Charnwood, versus approximately 600 with an explicit dependency or preceding fence. Artemisia native takes about 293–295 ticks independently, versus 430–432 dependent and 439–440 fenced. The matched code order, dependency change and common block boundary support younger ready arithmetic executing ahead of the older load-dependent multiply. A speedup from independent operations alone would not establish this ordering property.

Hot independent K=64 is substantially shorter than cold K=64 on both hosts; the older cold load provides an appreciable waiting interval. Artemisia's cold K=0 is close to independent K=64, consistent with hidden arithmetic. Charnwood's cold K=0 instead exceeds its independent K=64 time. Memory/frequency state and fence behavior therefore preclude an exact additive latency identity. The conclusion rests on matched K=64 dependencies and the actual assembly, not subtraction of an assumed load-plus-compute sum.

Artemisia SGX shows the same ordering of the K=64 controls, with larger cold times and more variation. This supports the same qualitative behavior; it does not attribute the extra time to one SGX mechanism. An earlier pilot without the common block-end fence is retained as a rejected cross-iteration assay, and the partial first Artemisia collection failed its placement gate. Neither is pooled with the accepted campaign.

Evidence and reconstruction. `results/q2-charnwood-private/` (270 rows) and `results/q2-artemisia-retry/` (540 rows). Source and reconstruction are `trackb/gen_kernels.py` and `report/make_report_artifacts.py`.

4 Q3: confirm branch prediction (10 points)

4.1 Method and observable signal

The conditional kernel reads one byte per decision from the same sequential 64 KiB array. The byte selects one of two equal-length assembly arms; each adds to an observable outcome count and executes one unconditional jump. The treatments are constant zero, constant one, alternating, and shuffled balanced 50/50 outcomes. Seeds 42 and 271828 provide separate shuffled patterns. Array addresses, footprint and iteration count stay fixed.

A branchless kernel reads the same patterns and computes the same counts with arithmetic. Its instruction stream is different, so it is a sensitivity control for outcome-dependent input effects rather than an identical-work baseline. The branch kernel is explicit assembly, and linked-code checks verify its conditional jump, arm lengths and absence of if-conversion. Each timed sample executes 1,048,576 decisions after full-pattern warm-up. A prediction-sensitive contrast should increase shuffled timing and native branch misses while leaving the branchless contrast small.

```
.Lloop_tb_q3_branch:
movzbl (%r10,%rax),%edx
test %edx,%edx
jz .Lzero_tb_q3_branch
add $1,%r8
jmp .Ljoin_tb_q3_branch
.Lzero_tb_q3_branch:
add $1,%r9
jmp .Ljoin_tb_q3_branch
.Ljoin_tb_q3_branch:
```

Listing 3: Actual conditional hot-loop excerpt from `tb_q3_branch`; both outcome arms remain in the linked native and enclave instruction stream.

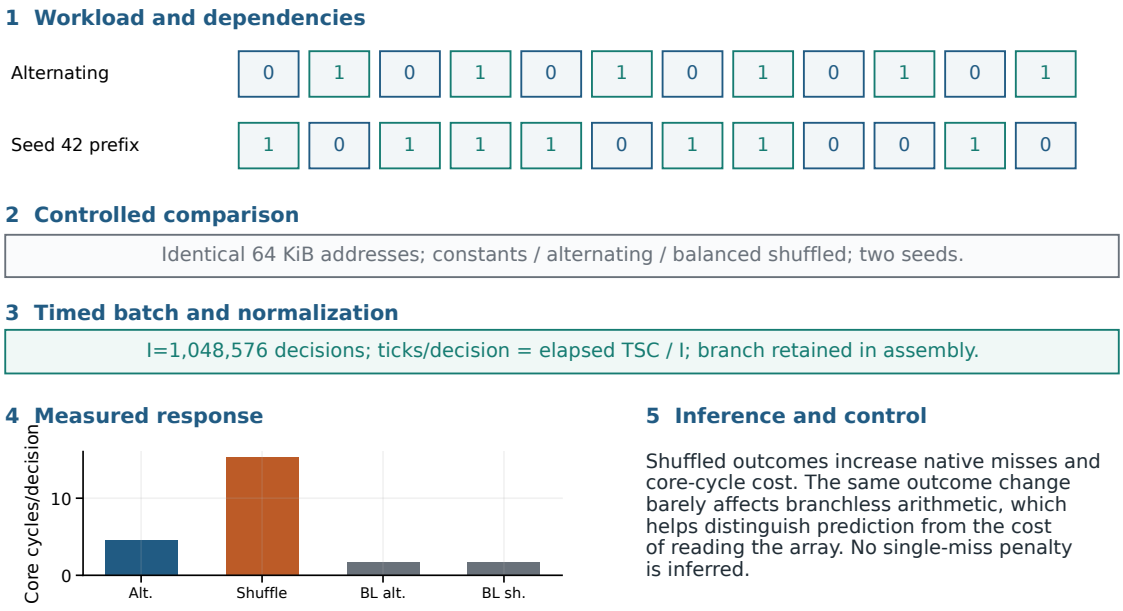


Figure 6: Q3's pattern and control. The shown shuffled prefix is generated with the recorded seed and 64 KiB shuffle procedure. The small response plot uses Artemisia native process 1; BL denotes the branchless kernel.

4.2 Native results, paired enclave result and inference

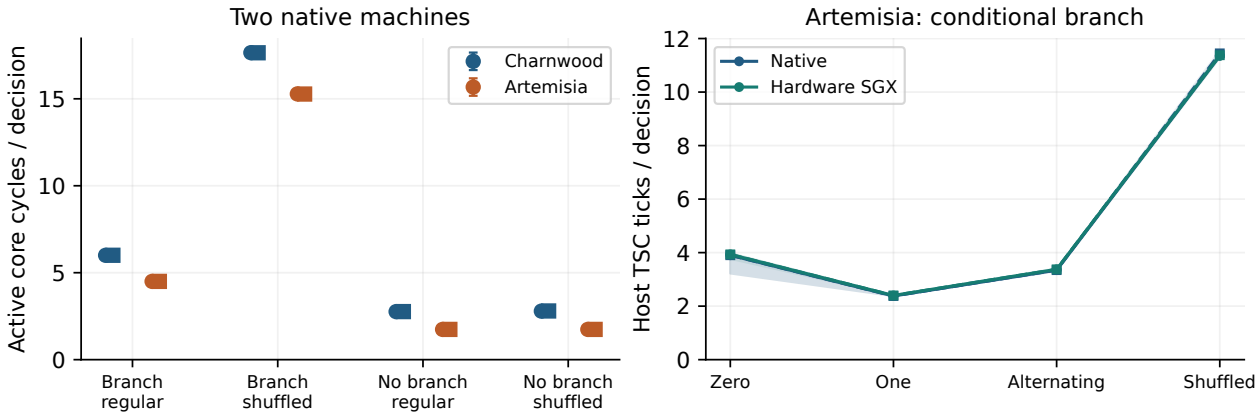


Figure 7: Q3 native core-cycle contrasts and Artemisia paired elapsed timings. The native comparison uses core cycles because Charnwood TSC costs drift between processes. All raw timing observations remain retained.

Pattern	Charnwood cycles/decision	Artemisia cycles/decision
Conditional, alternating	6.0015 / 6.0026	4.5009 / 4.5009
Conditional, shuffled 42	17.6524 / 17.6466	15.2814 / 15.2738
Conditional, shuffled 271828	17.6285 / 17.6237	15.2423 / 15.3203
Branchless, alternating	2.7628 / 2.7640	1.7397 / 1.7397
Branchless, shuffled 42	2.7921 / 2.8002	1.7397 / 1.7398

Table 6: Native Q3 medians in active core cycles/decision, R1 / R2. The random seed is part of each treatment label.

Pattern	Artemisia native ticks/decision	Artemisia SGX ticks/decision
Conditional, alternating	3.3457 / 3.3465	3.3767 / 3.3747
Conditional, shuffled 42	11.3628 / 11.4457	11.3539 / 11.3775
Conditional, shuffled 271828	11.3293 / 11.4062	11.3117 / 11.3375
Branchless, alternating	1.2936 / 1.2938	1.3156 / 1.3132
Branchless, shuffled 42	1.2941 / 1.2941	1.3151 / 1.3145

Table 7: Paired Artemisia Q3 medians in TSC ticks/decision, R1 / R2.

Figure 7 and Table 6 show both native CPUs predicting the regular stream successfully and incurring much higher cost for shuffled outcomes. Native shuffled misses are approximately 0.496–0.497 per decision on Charnwood and 0.469–0.473 on Artemisia, compared with only a few parts per million for alternating outcomes. The branchless pattern contrast remains small. Combined with the retained conditional jump, these observations support branch prediction; they do not measure a calibrated per-misprediction penalty or identify a predictor architecture.

Charnwood alternating conditional TSC medians drift from 5.5566 to 7.3413, and branchless alternating from 2.5604 to 3.9602, despite stable core-cycle costs. Elapsed timing alone would confound the comparison. SGX retains the regular/shuffled timing contrast, but counts only about 0.0003 branches/decision versus roughly 3 natively: protected miss-rate coverage is insufficient.

Evidence and reconstruction. `results/q3-charnwood-private/` (240 rows) and `results/q3-artemisia/` (480 rows). Generator and raw input hashes are in `report/generated/artifact-audit.json`.

5 Q4: ROB capacity through overlap loss (30 points)

5.1 Preserved method and observable signal

The completed Q4 implementation, observations, original plots and analyses are preserved. Each loop loads from two disjoint randomized pointer rings, with N single-byte NOPs between load A and load B and another N before the next load A. When the two dependency chains overlap, independent traversal is cheaper than a serialized traversal of the same complete line set. A long older miss prevents retirement; increasing filler spacing eventually prevents useful overlap across the finite instruction window.

The dependent control follows one cycle through all lines. Footprint is 256 MiB, stride 64 bytes, seed 42, one-byte NOPs and loop alignment 0. Each sample has 100,000 loops and two loads/loop. Charnwood's coarse sweep selected N=192..240 in steps of four; Artemisia's preserved fine range is 448..576. Each fine run has a fresh order-shuffled repeat. The 2N fillers, loads and loop control must not be conflated with an established total ROB entry count.

```
.Lloop_rob_independent_224_b1_a0:
mov (%rdi),%rdi
# 224 actual one-byte NOPs (0x90); repeated text omitted.
mov (%rsi),%rsi
# 224 further one-byte NOPs; repeated text omitted.
dec %rdx
jnz .Lloop_rob_independent_224_b1_a0
```

Listing 4: Actual unchanged `rob_independent_224_b1_a0` loop, with repeated NOP text explicitly omitted. Both native and enclave link the same kernel object.

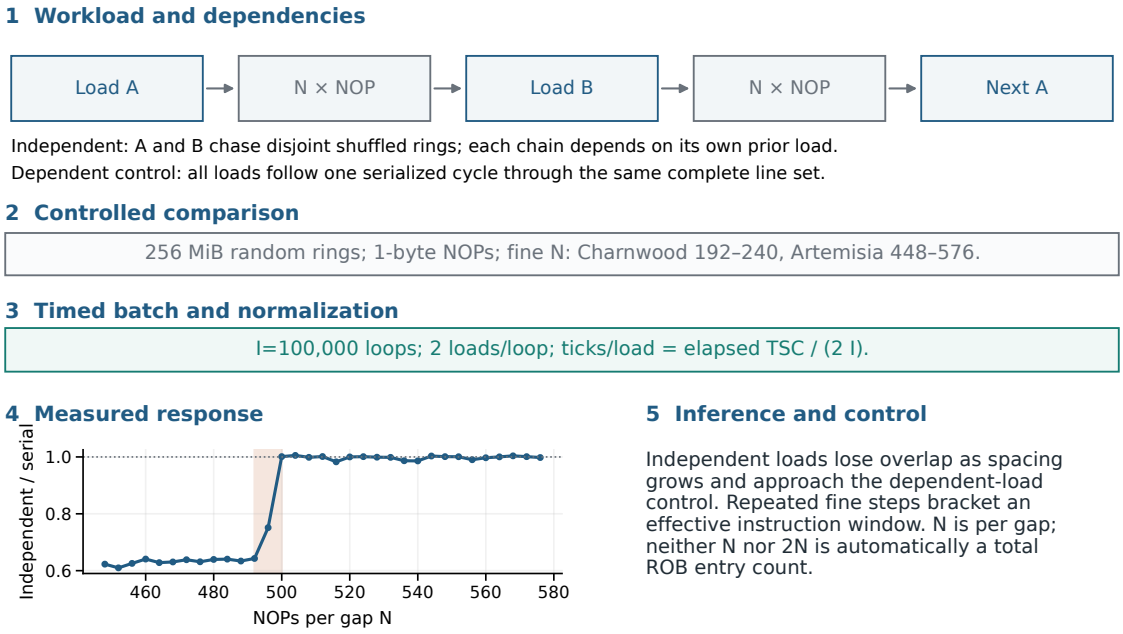


Figure 8: Q4's two-load spacing assay and serialized control. The small ratio plot is preserved Artemisia native process 1, reconstructed from raw observations without changing their original analysis.

5.2 Two-machine result and paired native/enclave result

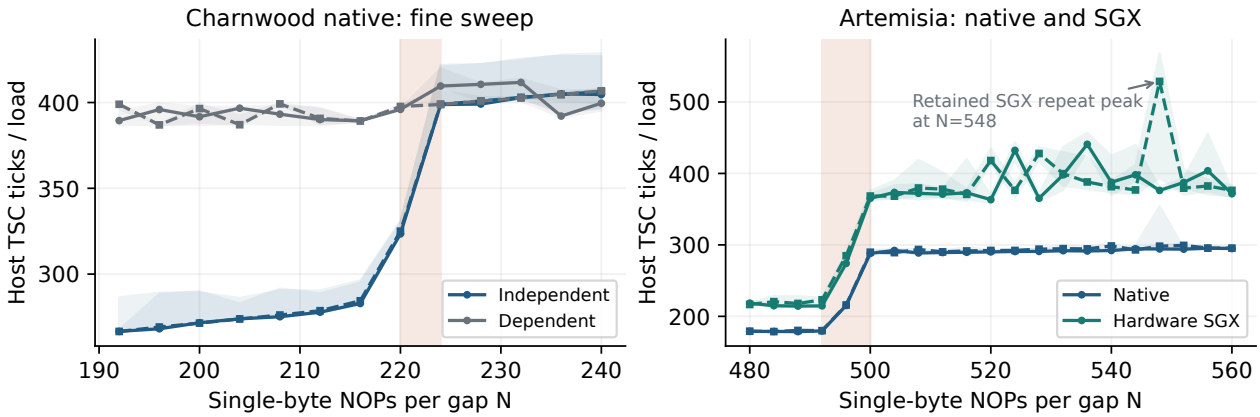


Figure 9: Q4 fine observations. Charnwood native shows independent and dependent traversals; Artemisia shows independent native/SGX traversals over a labeled zoom of the preserved range. Orange bands locate the reported operational transitions. The N=548 SGX-repeat peak is retained.

Charnwood N	Native ticks/load	Artemisia N	Native ticks/load	SGX ticks/load
216	282.84 / 284.44	492	179.55 / 180.02	214.67 / 223.14
220	323.44 / 325.03	496	215.28 / 215.96	274.12 / 284.81
224	398.81 / 398.94	500	288.09 / 289.73	365.20 / 368.36

Table 8: Independent-load medians in TSC ticks/load, R1 / R2. Different N columns reflect each host's own fine bracket, not matched elapsed durations across CPUs.

In Figure 9, Charnwood's largest fine increase is N=220→224 in both processes, also under minimum and unflagged-median diagnostics. At N=224, independent/dependent median ratios are about 0.974 and 1.000. Artemisia's preserved transition spans N=492–500, with native and hardware-SGX overlap loss in the same region. These are effective instruction-window bounds for the actual workload. They demonstrate a larger operational window on Artemisia, but exact physical ROB-entry attribution remains unproved.

The enclave's higher absolute ticks coexist with a similar principal transition; the evidence supports a shared qualitative window rather than a calibrated SGX cost per ROB entry. The isolated N=548 repeat peak would mislead a global largest-jump-only estimator. Both processes and all primary samples remain visible, so the transition is not sharpened by trimming that peak. Uncontrolled SMT on preserved Artemisia Q4 and Charnwood's legacy helper placement qualify the comparison.

No Artemisia Q4 rerun was needed. The user-confirmed missing Charnwood comparison was collected from unchanged source/build ID `bcd9d621feed36f6`. Its original 3,084-kernel checks and independent compiled native timer inspection passed. Charnwood retains 1,950 coarse rows and 390/390 fine rows; the Artemisia fine campaigns retain 1,980 rows each.

Evidence and reconstruction. `results/q4-coarse-charnwood/`, `results/q4-fine-charnwood/`, `results/q4-fine-repeat-charnwood/`, `results/v2-fine/` and `results/v2-fine-repeat/`. The preserved `ROB_BENCHMARK.md` and `ROB_RESULTS_ARTEMISIA.md` remain unchanged; new figure reconstruction is separate under `report/`.

6 Q5: integer physical-register allocation pressure (35 points)

6.1 Register class, method and observable signal

The tested class is **64-bit integer general-purpose registers (GPRs)**. We extend the two-ring layout by replacing NOP filler with `lea (reg,reg,2),reg`, which computes $3r$ modulo 2^{64} without a FLAGS destination. The filler avoids plain moves, zero idioms and pure constant-offset updates. Rotating one, four or eight accumulators changes dependency depth at the same destination-write count. All variants initialize the same eight GPR accumulators and preserve the same callee-saved registers.

A four-byte NOP control matches both filler count and encoded length; a serialized-load LEA control traverses the full same line set. While an older load prevents retirement, speculative destination writes create pressure on allocation and execution resources. An early multi-chain transition relative to the NOP window is evidence for integer-destination pressure; dependence and scheduler/throughput constraints remain competing limits. Each sample has 10,000 loops. Coarse N=0..512, step 16, selected fine N=120..160 on Charnwood and 192..232 on Artemisia, step four.

```
.Lloop_tb_q5_lea8_d0_208:
mov (%rdi),%rdi
lea (%r8,%r8,2),%r8
lea (%r9,%r9,2),%r9
# Rotate over all eight accumulators: 208 LEAs/gap.
mov (%rsi),%rsi
# 208 further scaled-index LEAs; repeated text omitted.
dec %rdx
jnz .Lloop_tb_q5_lea8_d0_208
```

Listing 5: Actual eight-chain scaled-index LEA kernel excerpt; repeated filler is marked as omitted. The eight accumulators are R8–R11, RBX, R12, R14 and R15.

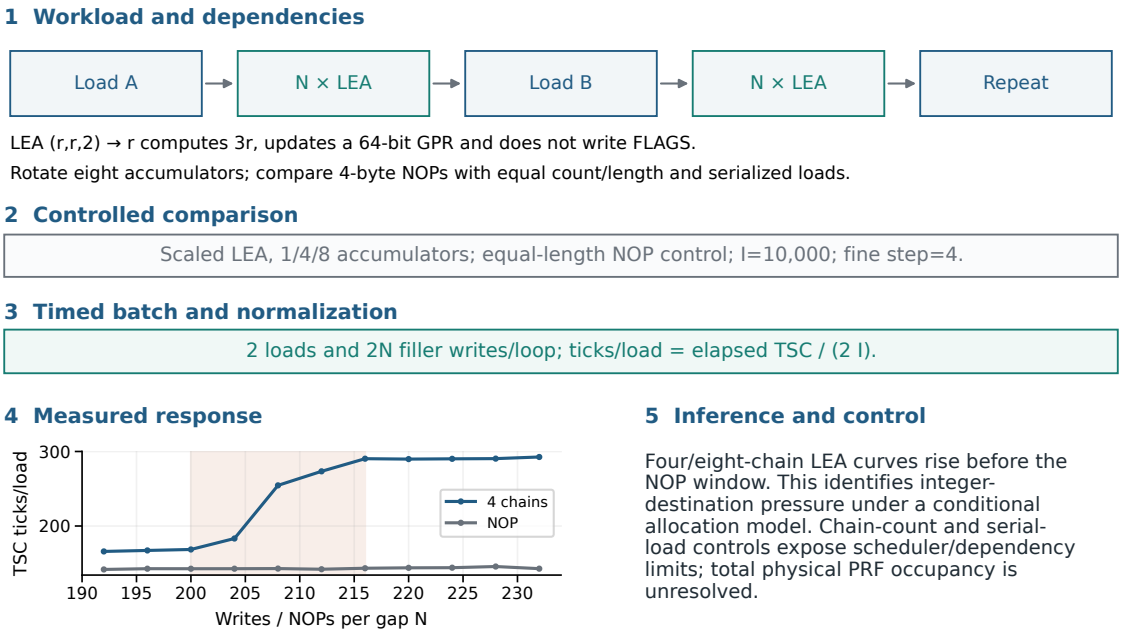


Figure 10: Q5's destination-pressure treatment. The small measured plot is Artemisia native process 1. The NOP control helps distinguish an earlier destination-sensitive limit from the later instruction-window limit.

6.2 Native bounds, enclave comparison and conversion limit

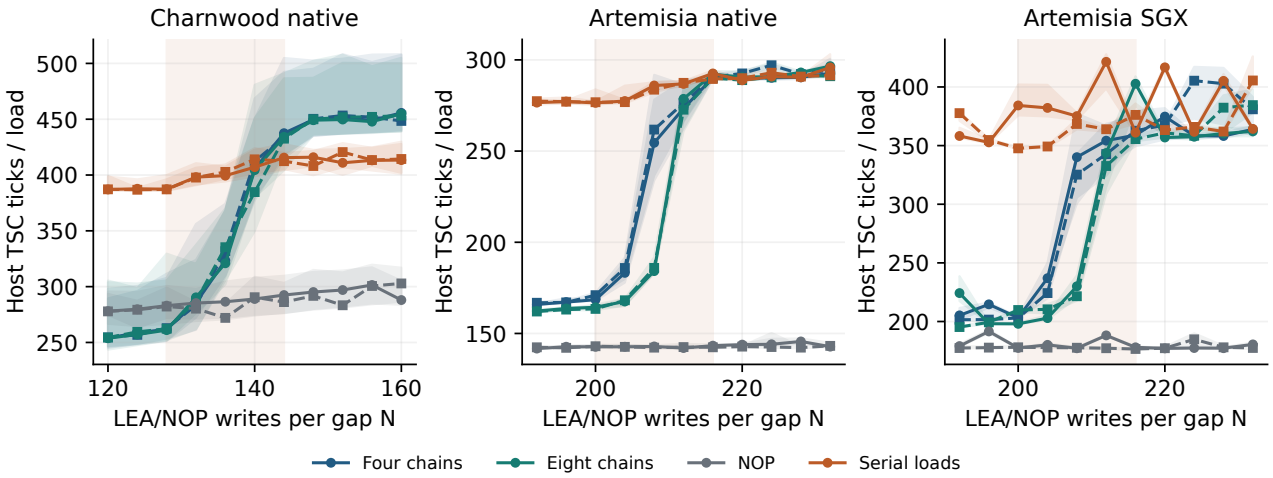


Figure 11: Q5 fine ranges chosen independently from each host's coarse observations. Native and SGX panels retain four/eight-chain, NOP and serialized-load controls. Process medians and observed p10–p90 remain separate; panel scales differ.

Host	N	Four LEA chains	Eight LEA chains	NOP control	Serial loads
Charnwood	128	261.9 / 262.8	261.0 / 262.2	283.0 / 282.2	387.3 / 387.1
Charnwood	136	323.4 / 331.8	320.9 / 335.3	286.5 / 272.0	399.4 / 402.8
Charnwood	144	437.5 / 434.0	434.2 / 432.6	292.5 / 286.2	415.6 / 412.4
Artemisia	200	168.5 / 171.0	164.4 / 163.5	142.7 / 143.0	276.4 / 276.8
Artemisia	208	254.5 / 261.8	184.2 / 186.0	142.9 / 142.2	286.0 / 283.7
Artemisia	216	290.3 / 290.6	292.2 / 290.0	143.3 / 142.4	292.6 / 289.5

Table 9: Q5 native process medians in TSC ticks/load. N is writes or NOPs per gap; R1 / R2 are separate processes.

N	Native four chains	SGX four chains	Native eight chains	SGX eight chains
200	168.5 / 171.0	203.2 / 203.0	164.4 / 163.5	198.0 / 209.8
208	254.5 / 261.8	340.1 / 325.3	184.2 / 186.0	230.0 / 221.5
212	273.4 / 276.3	354.2 / 342.8	278.6 / 272.7	343.7 / 332.6
216	290.3 / 290.6	359.0 / 362.7	292.2 / 290.0	402.7 / 355.5

Table 10: Paired Artemisia Q5 process medians in TSC ticks/load, R1 / R2.

Figure 11 shows Charnwood's four/eight-chain curves rising across roughly N=128–144 while NOPs remain low; Artemisia's corresponding region is approximately N=200–216. The NOP transitions occur much later, around coarse N=208–224 and 496–512, respectively. The direction survives the four/eight-chain change, supporting an integer-destination-sensitive operational bound. The onset shifts with chain count, and Charnwood's later LEA time can exceed its serialized control: allocation, dependency and scheduling/throughput effects are coupled.

Artemisia SGX has the same broad earlier region with greater variation, including the retained eight-chain process-1 N=216 value of about 403 ticks/load. Shared bytes do not ensure identical initial physical-register occupancy in the two environments.

6.3 Control separation and scope of the register estimate

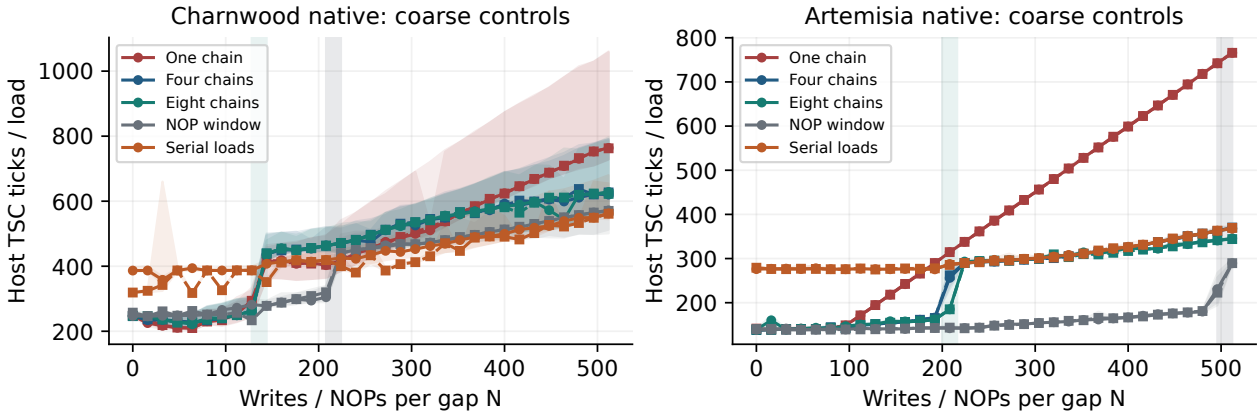


Figure 12: The complete Q5 native coarse sweeps retain the one-chain execution diagnostic, multi-chain treatments, serialized loads and NOP control. Green bands mark the multi-chain pressure region; gray bands mark the later NOP-window region. These are actual separate process medians, not schematic capacity curves.

Figure 12 shows why the fine interval was chosen from Q5's own coarse data instead of reusing the Q4 NOP bracket. Integer-destination work changes behavior substantially before NOPs exhaust the instruction window. The one-chain curve rises early because a long arithmetic dependency itself costs time; it is not a capacity estimate. Persistence across four and eight chains is stronger evidence, but execution resources and the scheduler can still limit how much of the older miss is hidden. The full-range controls therefore support the destination-sensitive bound without uniquely assigning it to a physical structure.

The first constant-offset `lea 1(reg),reg` campaign is retained separately: its checked results and unexpectedly short one-chain cycles invalidated the assumed ordinary allocation/execution model. Its mechanism was not established, and those measurements are not pooled with the revised scaled-index treatment.

What quantity is established. Conditional on allocation being the bottleneck, N=128–144 and N=200–216 bracket an effective speculative integer-destination budget. Precise PRF-versus-scheduler attribution and the conversion to *total* physical PRF size are unresolved. A conversion would require measured or justified committed/live mappings, other in-flight destinations, reclamation timing and reserved capacity. Neither $2N$ nor $N + 16$ supplies that accounting. The experiment is complete; its supported answer is a conditional bound rather than an invented total size.

Evidence and reconstruction. `results/q5-charnwood-private/`, `results/q5-fine-charnwood-private/`, `results/q5-scaled-residency-artemisia/` and `results/q5-fine-artemisia/`. The coarse/fine rows are 4,950/1,650 on Charnwood and 9,900/3,300 on Artemisia.

7 Q8: additional cost of an STLB-served TLB miss (10 points)

7.1 Miss class, baseline and controls

The target is an **L1 data-TLB miss served by the STLB**, rather than a page walk or an unspecified translation/cache mixture. Both layouts use the same node count, cache-line count, seed and random dependent traversal order. Compact nodes occupy consecutive 64-byte lines. Scattered node i occupies `base + i*4096 + (i%64)*64`: one line per 4 KiB page, spread over all 64 L1 set offsets. Both native and enclave arenas are page-aligned; hardware checks verify actual pointer geometry and cursors.

The 128/256/384 nodes use only 8/16/24 KiB of data in either layout. Full-ring warming helps keep data hot while scattered translations exceed the L1-TLB working set. The compact layout is the translation-resident baseline. Dependency prevents overlap of successive loads. Four simultaneous native counters record STLB services, completed 4 KiB walks, L1D misses and active core cycles, using Intel's model-specific event definitions [6, 7].

```
.Lloop_tb_q8_chase:
mov (%r8),%r8
dec %rsi
jnz .Lloop_tb_q8_chase
```

Listing 6: Actual dependent loop `tb_q8_chase`; address construction and warm-up are outside this timed loop.

```
/* From trackb/engine.c, node(i): */
if(cfg.question==8 && cfg.mode==1) return arena+i*4096+(i%64)*64;
return arena+i*64;
```

Listing 7: Actual layout arithmetic from `trackb/engine.c`. Compact and scattered layouts share the same randomized node order.

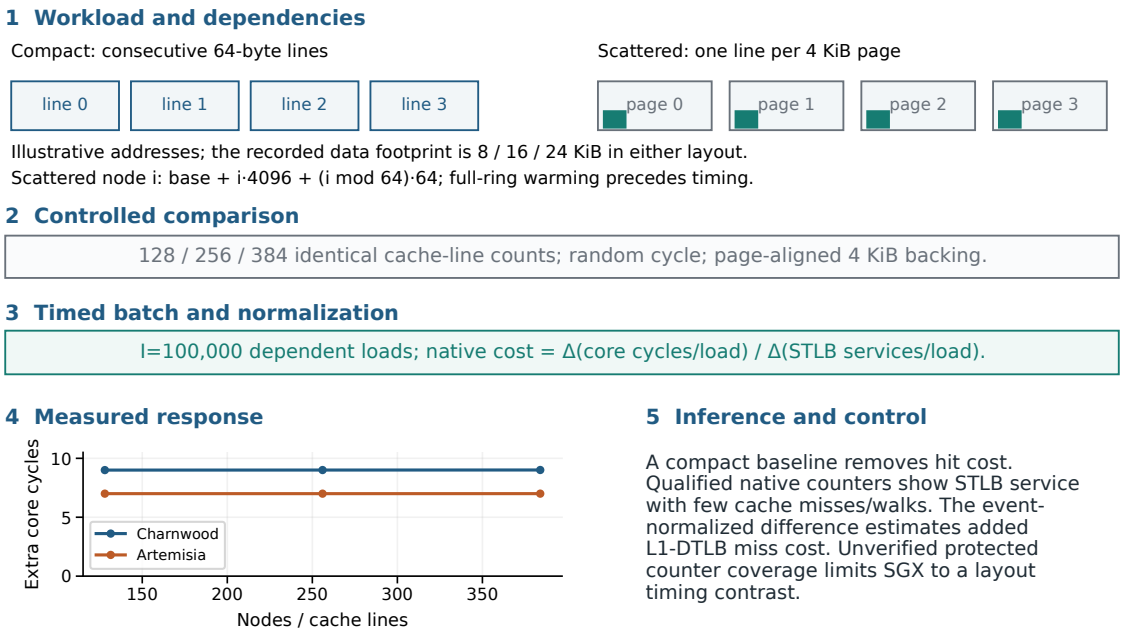


Figure 13: Q8 changes the translation working set while matching touched data lines. Address boxes are schematic; the small cost plot uses the actual class-qualified native process-1 estimates.

7.2 Qualified native cost and paired enclave observations

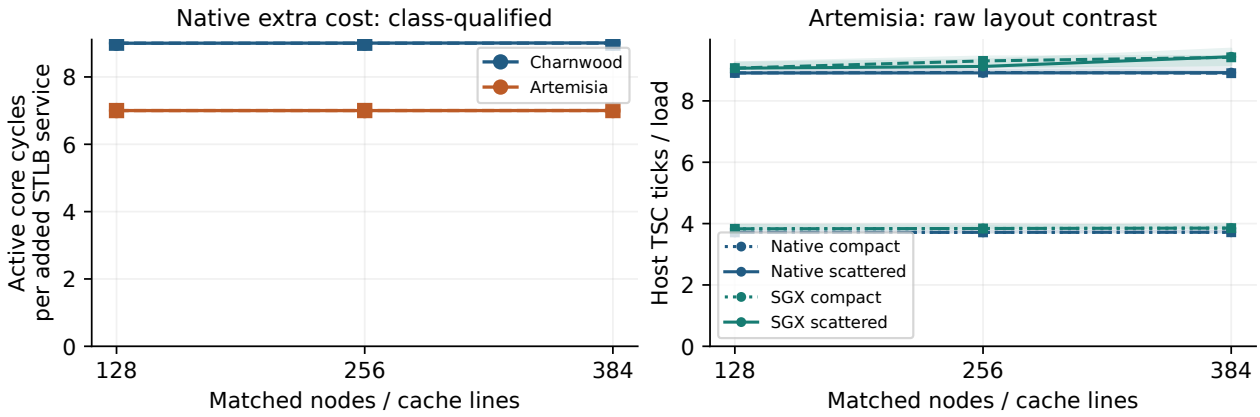


Figure 14: Left: native cost from separate process medians; no confidence interval is assigned to this derived ratio. Right: Artemisia compact (dotted) and scattered timing. Protected miss attribution remains unqualified.

Cache lines / nodes	Charnwood extra core cycles	Artemisia extra core cycles
128	9.0014 / 9.0029	7.0016 / 7.0015
256	9.0052 / 9.0014	7.0007 / 7.0023
384	9.0101 / 9.0043	7.0024 / 7.0020

Table 11: Additional active core cycles per added STLB service, R1 / R2.

Nodes	Native compact	Native scattered	SGX compact	SGX scattered
128	3.7132 / 3.7131	8.9105 / 8.9113	3.8293 / 3.8301	9.0643 / 9.0690
256	3.7143 / 3.7143	8.9119 / 8.9268	3.8404 / 3.8404	9.1226 / 9.3032
384	3.7178 / 3.7181	8.9222 / 8.9026	3.8518 / 3.8529	9.4392 / 9.4201

Table 12: Artemisia raw layout medians in TSC ticks/load, R1 / R2.

For Figure 14 and Table 11, each matched process uses the estimate

$$C_{\text{extra}} = \frac{\text{median}(c_{\text{scattered}}/I) - \text{median}(c_{\text{compact}}/I)}{\text{median}(s_{\text{scattered}}/I) - \text{median}(s_{\text{compact}}/I)},$$

where c counts active core cycles and s counts STLB services. Require native coverage $\geq 98\%$, added services > 0.5/load, compact services < 0.05/load, and both walks and L1D misses < 0.01/load in either layout. Both hosts pass: recorded 4 KiB walks are zero, and maximum L1D misses/load are 0.00273 on Charnwood and 0.0008 on Artemisia. The supported extra costs are therefore about **9 and 7 active core cycles**, above the compact baseline. Page-walk costs are not measured; counter semantics limit precision.

TSC cost also requires effective cycle/TSC medians matching within 2% and each condition's p90/p10 ratio ≤ 1.02. All six Artemisia native pairs pass (about 5.20 ticks/service); four of six Charnwood TSC estimates are withheld for drift/mismatch. Earlier shifted baselines remain documented. SGX layouts differ in timing, but unqualified protected counters limit that result to a translation-working-set contrast rather than a verified miss cost.

Evidence and reconstruction. `results/q8-clock-charnwood-private/` (180 rows) and `results/q8-clock-artemisia/` (360 rows). Final derived quantities and qualification text are generated in `report/generated/q8-contrasts.csv`.

8 Limitations, reproducibility and disclosure

8.1 What the evidence does and does not resolve

All six selected experiments have completed repeated native campaigns on both machines and corresponding Artemisia hardware-enclave campaigns. Completion of collection does not make every inferred physical quantity exact. The principal inference limits are specific:

- Q1's execution-unit resource bound comes from a cited model, not an experimental unit count; Q2's dependency/fence evidence supports ordering without instruction tracing.
- Q4 is an operational overlap-loss window, with uncontrolled SMT in its preserved Artemisia runs. Q5 identifies integer GPR pressure, with unresolved precise PRF attribution and total-size conversion.
- Q8's native class/cache/cycle criteria support its specific STLB-served miss cost. Protected event visibility is insufficient to transfer that attribution to SGX. Full TLB hierarchy reconstruction is not selected.
- PMU brackets differ slightly from TSC brackets. Frequency, kernel/IRQ/AEX activity, physical EPC placement and other cores' shared-memory traffic remain incompletely controlled. Context-switch/fault flags are observations, not complete interrupt tracing.

The failed or rejected paths remain visible: the early cross-iteration Q2 prototype; stopped NUMA-accounting/SMT attempts; the constant-offset LEA allocation diagnostic; and the three-counter Q8 compact-baseline shift. They explain methodological choices without replacing the final evidence. The only requested setup artifact not located is the original SampleEnclave console log; the user confirmed it was not saved. Its success is explicitly identified as user-confirmed history, while custom hardware validation logs and benchmark runtime status are retained. No missing result is filled from published specifications.

8.2 Evidence map and reproduction

The repository is https://github.com/vvenka25/ECE-592-Project-3. Its `REPRODUCING.md` describes the hash-checked restoration of `data/raw-results.tar.gz`, which restores recorded manifests, raw CSVs, mapping audits and source snapshots under `results/`. Campaign manifests give commands, parameters, process seeds, CPU/node requests, source/binary hashes, failures and disturbances. Each question's evidence paragraph names the actual campaign. The report generator's input hashes and code-excerpt provenance are in `report/generated/artifact-audit.json` and `report/generated/source-excerpts.json`. No Project 1 content is an input to the figure/table generator.

From the project root, restore data if using the public checkout and rebuild this report's numerical tables, vector figures and PDF:

```
python3 data/unpack_results.py   # public checkout only; local data already present
python3 -m pip install -r requirements.txt
MPLCONFIGDIR=/tmp/hw3-mpl python3 report/make_report_artifacts.py
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=report report/report.tex
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=report report/report.tex
```

The first command is not needed in the original workspace, where the raw files are already present. The local dependency is Matplotlib 3.9.4; plain pdfLaTeX builds the supplied figures and tables without executing Python. Historical analysis revisions remain alongside existing plots.

Fresh collection is distinct from regenerating figures. Recheck allowed CPUs, NUMA nodes and SMT state; use fresh output directories. These examples show new-suite validation and a planned Q3 run, not a request to repeat the completed campaigns:

```
make -C trackb native
TEST_CPU=12 make -C trackb check
source /opt/intel/sgxsdk/environment
make -C trackb native enclave
TEST_CPU=12 TEST_SGX=1 make -C trackb check
python3 trackb/run.py --question Q3 --cpu 12 --numa-node 0 \
  --environment both --require-idle-sibling --out results/new-q3
# Inspect the plan, then add --execute to collect new observations.
```

On Charnwood, the collection helper `report/collect_charnwood.sh` records private core scheduling and controller isolation. Follow Q3→Q1→Q2→Q5→Q8 for a new collection, and choose Q5 fine bounds from that host's own coarse observations. Use the isolated `report/reproduce_q4_charnwood.py` for unchanged Q4 source; it refuses execution on Artemisia. Exact historical builds use each campaign's own source snapshot and toolchain. Current-source reruns are separate evidence.

The generated suite contains 1,053 functions; retained validation checks semantics, shared native/enclave bytes, native/enclave-host timer assembly, Q8 page geometry, denied-PMU behavior and native page queries. Native functional checks do not imply hardware SGX execution: the latter requires `TEST_SGX=1` and is documented in `results/trackb-four-counter-hardware-check.log`. Q4's preserved 3,084-function checks and Charnwood timer audit are separate.

8.3 Overleaf, submission files and AI-use disclosure

The user-designated project is https://www.overleaf.com/project/6aeeb80e5d4ccedd231a327b. The accompanying Overleaf source ZIP includes `main.tex`, generated LaTeX tables, vector figures, source excerpts and build instructions. It compiles without fetching repository paths or running analysis online. The normal project URL is included; the private sharing token is omitted. No successful direct upload is claimed without an authenticated editing connection. Upload/import of the source ZIP remains a separate publishing step if such a connection is unavailable.

The assignment requires a ZIP containing the PDF report, source, raw data, plotting scripts and reproduction instructions, plus an AI-use log for Track B [1]. The revised submission includes those groups and the original evidence. The earlier submission and Q4 preservation archive remain intact; the revised report and new figures are separate artifacts. GitHub's source-only preservation derivative excludes private signing keys and generated binaries while retaining the original non-build bytes.

Codex assisted with benchmark/code/test design, hardware collection, methodological revisions, analysis, figures and report drafting, as recorded chronologically in `AI_USE_LOG.md`. For this revision it read the Project 1 PDF only to guide format, structure and explanation style, read `Homework_3.pdf` for requirements, and generated fresh diagrams and tables from Homework 3 sources and data. It did not copy Project 1 prose, measurements, code, artwork, conclusions or assignment-specific phases. No new performance campaign was run for this writing revision. Synthetic checks are labeled engineering tests, not measurements. The author must understand and be able to explain the submitted code and conclusions.

References

[1] ECE 492/592, *Homework 3: Microarchitecture Reverse Engineering Across Machines*, Fall 2026. Supplied `Homework_3.pdf`, pp. 2–4 (Track B selection and common submission requirements).

[2] Intel, *Intel Software Guard Extensions Web-Based Training*, especially application partitioning and debug enclaves. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sgx-web-based-training.html. Accessed November 7, 2026.

[3] LLVM, *Skylake Client scheduling model*, `WriteIMul64Imm` resource. https://raw.githubusercontent.com/llvm/llvm-project/main/llvm/lib/Target/X86/X86SchedSkylakeClient.td. Accessed November 7, 2026.

[4] LLVM, *Sapphire Rapids scheduling model*, `WriteIMul64Imm` resource. https://raw.githubusercontent.com/llvm/llvm-project/main/llvm/lib/Target/X86/X86SchedSapphireRapids.td. Accessed November 7, 2026.

[5] Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual, Instruction Set Reference*, LFENCE. https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[6] Intel PerfMon, *Skylake core event definitions*, DTLB load-miss/STLB-hit, completed walks and retired L1D misses. https://raw.githubusercontent.com/intel/perfmon/main/SKL/events/skylake_core.json. Accessed November 7, 2026.

[7] Intel PerfMon, *Sapphire Rapids core event definitions*. https://raw.githubusercontent.com/intel/perfmon/main/SPR/events/sapphirerapids_core.json. Accessed November 7, 2026.

[8] Linux kernel documentation, *Core Scheduling*. https://docs.kernel.org/admin-guide/hw-vuln/core-scheduling.html.

[9] Linux man-pages project, *move_pages(2)*. https://man7.org/linux/man-pages/man2/move_pages.2.html.